

Лабораторная работа № 14

Программирование в командном процессоре ОС UNIX. Ветвления и циклы
(продолжение)

Пириев Керем

2026-10-05

Содержание I

- 1 Информация
- 2 Цель работы
- 3 Задание
- 4 Выполнение лабораторной работы
- 5 Выводы

Раздел 1

Информация

- Пириев Керем
- студент группы НПИбд-02-25
- Российский университет дружбы народов
- 1032254162@rudn.ru

Раздел 2

Цель работы

Цель работы

Изучение основ программирования в оболочке ОС UNIX, написание сложных командных файлов с использованием логических конструкций, циклов и механизмов синхронизации.

Раздел 3

Задание

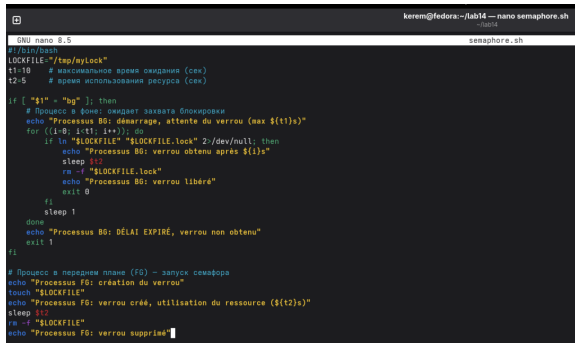
- 1 Создание рабочего каталога `~/lab14`.
- 2 Разработка скрипта семафоров `semaphore.sh`.
- 3 Разработка аналога команды `man` — `my_man.sh`.
- 4 Разработка генератора случайных букв `random_letters.sh`.

Раздел 4

Выполнение лабораторной работы

Создание рабочего каталога и semaphore.sh

Разработка скрипта синхронизации процессов через блокировки:



```
kerem@fedora:~/lab14 — nano semaphore.sh
~lab14
semaphore.sh

GNU nano 8.5
#!/bin/bash
LOCKFILE="/tmp/myLock"
t1=10 # максимальное время ожидания (сек)
t2=5  # время использования ресурса (сек)

if [ "$1" = "bg" ]; then
    # Процесс в фоне: ожидает захвата блокировки
    echo "Processus BG: démarrage, attente du verrou (max ${t1}s)"
    for ((i=0; i<t1; i++)); do
        if ln "$LOCKFILE" "$LOCKFILE.lock" 2>/dev/null; then
            echo "Processus BG: verrou obtenu après ${i}s"
            sleep $t2
            rm -f "$LOCKFILE.lock"
            echo "Processus BG: verrou libéré"
            exit 0
        fi
        sleep 1
    done
    echo "Processus BG: DÉLAI EXPIRÉ, verrou non obtenu"
    exit 1
fi

# Процесс в переднем плане (FG) – запуск семафора
echo "Processus FG: création du verrou"
touch "$LOCKFILE"
echo "Processus FG: verrou créé, utilisation du ressource (${t2}s)"
sleep $t2
rm -f "$LOCKFILE"
echo "Processus FG: verrou supprimé"
```

Рисунок 1: Скрипт semaphore.sh в nano

Запуск semaphore.sh

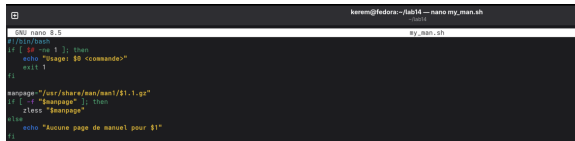
Проверка работы механизма таймаута и ожидания ресурса:

```
kerem@fedora:~/lab14$ cd ~/lab14
kerem@fedora:~/lab14$ chmod +x semaphore.sh
kerem@fedora:~/lab14$ ./semaphore.sh
Processus FG: création du verrou
Processus FG: verrou créé, utilisation du ressource (5s)
Processus FG: verrou supprimé
kerem@fedora:~/lab14$
```

Рисунок 2: Запуск semaphore.sh

Скрипт my_man.sh (Создание)

Написание скрипта для поиска и чтения страниц руководства:



```
kerem@fedora:~/lab14 — nano my_man.sh
GNU nano 3.5 my_man.sh
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <command>"
    exit 1
fi

manpage="/usr/share/man/man1/${1}.gz"
if [ -f "$manpage" ]; then
    zless "$manpage"
else
    echo "Aucune page de manuel pour $1"
fi
```

Рисунок 3: Скрипт my_man.sh

Скрипт my_man.sh (Тест)

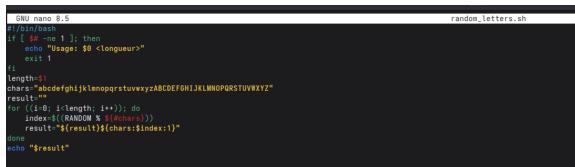
Проверка поиска и вывода справки для команды ls:

```
kerem@fedora: ~/lab14
~lab14
.\ " DO NOT MODIFY THIS FILE! It was generated by help2man 1.50.1.
.TH LS "1" "September 2025" "GNU coreutils 9.7" "User Commands"
.SH NAME
ls \- list directory contents
.SH SYNOPSIS
.B ls
[\fI]\fI,\fIOPTION\fI... [\fI]\fI,\fIFILE\fI...
.SH DESCRIPTION
.\ " Add any additional description here
.PP
List information about the FILES (the current directory by default).
Sort entries alphabetically if none of \fB-\fR-cftuvSUX\fR nor \fB-\fR-sort\fR is specified.
.PP
Mandatory arguments to long options are mandatory for short options too.
.TP
\fB-\fR-a\fR, \fB-\fR-all\fR
do not ignore entries starting with .
.TP
\fB-\fR-A\fR, \fB-\fR-almost-all\fR
do not list implied . and ..
.TP
\fB-\fR-author\fR
with \fB-\fR-l\fR, print the author of each file
.TP
\fB-\fR-b\fR, \fB-\fR-escape\fR
print C\style escapes for nongraphic characters
.TP
\fB-\fR-block-size=\fR\fI\fR,\fISIZE\fI
with \fB-\fR-l\fR, scale sizes by SIZE when printing them;
e.g., \fB-\fR-block-size=M\fR; see SIZE format below
.TP
\fB-\fR-B\fR, \fB-\fR-ignore-backups\fR
do not list implied entries ending with ~
.TP
\fB-\fR-c\fR
with \fB-\fR-lt\fR: sort by, and show, time (time of last
change of file status information);
with \fB-\fR-l\fR: show ctime and sort by name;
otherwise: sort by ctime, newest first
.TP
\fB-\fR-C\fR
```

Рисунок 4: Тестирование my_man.sh

Скрипт random_letters.sh (Создание)

Разработка генератора случайных строк на базе \$RANDOM:



```
GNU nano 8.5 random_letters.sh
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <longueur>"
    exit 1
fi
length=$1
chars="abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
result=""
for ((i=0; i<length; i++)); do
    index=$((RANDOM % ${#chars}))
    result="${result}${chars:index:1}"
done
echo "$result"
```

Рисунок 5: Скрипт random_letters.sh в nano

Скрипт random_letters.sh (Тест)

Генерация случайной последовательности из 10 символов:

```
kerem@fedora:~/lab14$ ./my_man.sh ls
kerem@fedora:~/lab14$ nano random_letters.sh
kerem@fedora:~/lab14$ chmod +x random_letters.sh
kerem@fedora:~/lab14$ ./random_letters.sh 10
bAekrbXifB
kerem@fedora:~/lab14$
```

Рисунок 6: Запуск random_letters.sh

Раздел 5

Выводы

- Разработаны скрипты для демонстрации межпроцессного взаимодействия через семафоры и блокировки.
- Реализован собственный аналог утилиты `map` для работы со сжатыми файлами справки.
- Освоено использование переменной `$RANDOM` и строковых подстановок для генерации случайных данных в `bash`.